

项目简介

PAM 是一个基于 Java 的宠物领养管理系统，旨在为流浪动物寻找温暖的家，同时为管理员提供高效的领养管理工具。系统支持用户注册登录、浏览宠物、提交领养申请、查看公告等功能，管理员可以管理宠物信息、审核申请、发布公告等。

当前架构状态: 胖客户端 (Fat Client) 架构

- 所有业务逻辑在客户端进程运行
- 客户端直接连接 MariaDB 数据库 (localhost:3306)
- Server 端 (PAMServer 和 ClientHandler) 目前为空壳未实现
- 未来方向: 真正的 C/S 架构需通过 TCP Socket 通信, 由 Server 统一处理业务逻辑

核心功能

用户端功能

已实现功能

1. 用户认证与账户管理

- ☒ 用户注册 (用户名、密码、姓名、手机、地址)
- ☒ 用户登录 (权限验证, 区分普通用户和管理员)
- ☒ 退出登录
- ☒ 注销账户 (级联删除用户数据)
- ☒ 修改个人信息 (手机号、地址)
- ☒ 修改密码

2. 宠物浏览与申请

- ☒ 查看所有可领养宠物
- ☒ 查看宠物详情 (名称、物种、年龄、描述、状态)
- ☒ 提交领养申请
- ☒ 查看我的申请列表
- ☒ 查看申请详情及审核状态

3. 公告系统

- ☒ 查看所有公告列表
- ☒ 查看公告详情（标题、内容、发布时间）
- ☒ 按时间排序显示

4. 图形界面 (Swing)

- ☒ 菜单栏导航（浏览宠物、我的申请、系统公告、个人信息、账户）
- ☒ 主窗口展示结果
- ☒ 对话框交互（登录、注册、输入、确认）
- ☒ 友好的错误提示和成功消息

5. 命令行界面 (CLI)

- ☒ 基于 JLine 3 的交互式终端
- ☒ 菜单驱动操作
- ☒ 密码隐藏输入

未来开发方向

1. 用户体验增强

-  宠物收藏/关注功能
-  申请进度实时推送
-  在线客服聊天
-  图片上传（宠物照片、用户头像）
-  宠物搜索和筛选（物种、年龄、状态）
-  分页加载大量数据

2. 社交功能

-  寻宠公告专区
-  志愿者招募报名
-  爱心物资捐赠通道
-  用户评价和反馈

3. 移动端适配

-  Android/iOS App
 -  响应式 Web 界面
-

■ 已实现功能

1. 管理员认证

- ☒ 管理员登录（权限验证）
- ☒ 退出登录

2. 宠物管理

- ☒ 查看所有宠物（包括已领养）
- ☒ 添加新宠物（名称、物种、年龄、描述）
- ☒ 修改宠物信息
- ☒ 删除宠物
- ☒ 宠物统计（总数、按状态分类）

3. 领养申请审核

- ☒ 查看所有申请
- ☒ 查看指定申请详情
- ☒ 处理待审核申请（接受/拒绝/搁置）
- ☒ 添加审核意见
- ☒ 申请统计（总数、待审核、已通过、已拒绝）

4. 公告管理

- ☒ 发布新公告
- ☒ 查看所有公告
- ☒ 查看公告详情
- ☒ 修改公告
- ☒ 删除公告
- ☒ 筛选公告（按状态）

5. 用户管理

- ☒ 查看所有用户
- ☒ 删除用户账户
- ☒ 用户统计（总数、管理员数、普通用户数）

6. 系统统计

- ☒ 综合统计概览（用户、宠物、申请）

7. 命令行界面

- ☒ 多级菜单系统

-  交互式输入处理
-  权限验证

未来开发方向

1. 管理功能增强

-  批量操作（批量导入宠物、批量审核）
-  数据导出（Excel、CSV）
-  高级搜索和过滤
-  操作日志审计

2. 数据分析

-  领养趋势图表
-  用户活跃度分析
-  热门宠物类型统计
-  月度/年度报告生成

3. 自动化

-  自动审核规则引擎
-  定时任务（清理过期数据、发送提醒）
-  智能推荐匹配宠物

技术架构

分层架构设计

```
1  Client (UI层)
2  ↓
3  ServerApi (API层 - 统一入口)
4  ↓
5  Service (业务逻辑层)
6  ↓
7  Manager (管理层 - 单例模式)
8  ↓
9  DAO (数据访问层)
10 ↓ Database (MariaDB)
```

技术栈

- 后端框架: Java 21
- 数据库: MariaDB 10.x
- ORM: Hibernate/JPA
- 终端交互: JLine 3
- GUI: Java Swing
- 日志: SLF4J + Logback
- JSON处理: Gson
- 构建工具: Maven
- 加密: SHA-256 + Base64

设计模式

- 单例模式: UserManager, PetManager, AnnouncementManager, AdoptionApplicationManager
- Result 模式: 统一封装操作结果 (success/error)
- Builder 模式: 实体类构建
- 异常驱动: BusinessException 携带错误码

项目结构

PAM/

```
├── src/
│   ├── client/ # 客户端代码
│   │   ├── Client.java # 客户端基类
│   │   ├── UserClient.java # 用户命令行客户端
│   │   ├── UserUI.java # 用户图形界面客户端
│   │   └── AdminClient.java # 管理员命令行客户端
│   │
│   ├── server/ # 服务端代码 (待实现)
│   │   ├── api/ # API层
│   │   │   └── ServerApi.java # 统一API入口
│   │   ├── service/ # 业务逻辑层
│   │   │   ├── UserService.java
│   │   │   └── PetService.java
```

```
| | | ├── AnnouncementService.java
| | | └── AdoptionApplicationService.java
| | └── handler/ # 请求处理器 (空)
| |
| ├── common/ # 公共模块
| | ├── entity/ # 实体类
| | | ├── User.java
| | | ├── Pet.java
| | | ├── Announcement.java
| | | └── AdoptionApplication.java
| | ├── manager/ # 管理层
| | | ├── UserManager.java
| | | ├── PetManager.java
| | | ├── AnnouncementManager.java
| | | └── AdoptionApplicationManager.java
| | ├── dao/ # 数据访问层
| | | ├── UserDAO.java
| | | ├── PetDAO.java
| | | ├── AnnouncementDAO.java
| | | └── AdoptionApplicationDAO.java
| | ├── enums/ # 枚举类型
| | ├── dto/ # 数据传输对象
| | ├── exception/ # 自定义异常
| | └── utils/ # 工具类
| | ├── PasswordEncoder.java
| | ├── PasswordGenerator.java
| | ├── Menu.java
| | ├── Validator.java
| | └── StringFormatter.java
| |
| ├── config/ # 配置文件
| | └── validator.json # 验证规则配置
| |
| └── test/ # 测试数据
| └── test_data.sql
|
└── doc/ # 文档和SQL脚本
    ├── pets.sql
    └── announcements.sql
```

| └─ Sqls.md
|
├─ target/ # 编译输出
├─ pom.xml # Maven配置
└─ README.md



快速开始

环境要求

- JDK 21+
- Maven 3.6+
- MariaDB 10.x
- IDE (推荐 IntelliJ IDEA)

安装步骤

1. 克隆项目

```
1 git clone https://github.com/sympsel/PAM.git
2 cd PAM
```

Fence 2

2. 配置数据库

```
1 CREATE
2 DATABASE pam_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Fence 3

3. 创建数据表

- 使用 `doc/Sqls.md` 中的建表语句

4. 生成管理员密码

```
1 -- 根据引导得到管理员创建的 sql 代码，需要 maven 环境，请检查环境变量
2 mvn compile java -cp target/classes common.utils.PasswordGenerator
  admin123
```

Fence 4

```
1 insert into users (id, username, password_hash, permission,  
  real_name, is_online, create_time)  
2 values (uuid(), 'admin', '加密后的密码', 'ADMIN', '管理员', false,  
  unix_timestamp() * 1000);
```

Fence 5

5插入测试数据

- 使用 `doc/announcements.md` 中的建表语句，确保senderId为已存在的管理员 ID，比如替换为建管理员创建得到的 ID
- 使用 `doc/pet.md` 中的建表语句

7. 运行客户端

- 图形界面: `java -cp target/classes client.UserUI`
- 命令行用户端: `java -cp target/classes client.UserClient`
- 命令行管理端: `java -cp target/classes client.AdminClient`



数据库设计

核心表结构

1. `users` - 用户表

- id, username, password_hash, permission, real_name, phone, address, is_online, create_time

2. `pets` - 宠物表

- id, name, specie, age, description, adoption_status, create_time

3. `adoption_applications` - 领养申请表

- id, applicator_id, pet_id, status, review_comment, reviewer_id, review_time, create_time

4. `announcements` - 公告表

- id, sender_id, title, content, create_time

外键约束

- `fk_application_applicator`: applications.applicator_id → users.id (CASCADE)
- `fk_application_pet`: applications.pet_id → pets.id (CASCADE)

- `fk_announcement_sender` : `announcements.sender_id` → `users.id` (CASCADE)
-

安全特性

密码加密

- 算法: SHA-256 + Base64
- 工具: `PasswordEncoder` 类
- 测试模式: 可通过 `validator.json` 配置开关

权限控制

- **ADMIN**: 管理员权限, 可执行所有管理操作
- **NORMAL**: 普通用户权限, 仅可浏览和申请

会话管理

- 在线状态追踪 (`is_online` 字段)
 - 防止重复登录
 - 登出时更新状态
-

开发规范

代码规范

- 变量命名: camelCase
- 常量命名: UPPER_SNAKE_CASE
- 类名: PascalCase
- SQL关键字: 小写
- 实体类: 使用 Lombok 注解

日志规范

- 使用 SLF4J + Logback
- 日志级别: INFO (生产), DEBUG (开发)
- 敏感信息不记录明文密码

异常处理

- Service 层抛出 BusinessException
- ServerApi 层统一捕获并返回 Result
- Client 层只判断 isSuccess()



测试

测试账号

```
1      用户名 / 密码
2  管理员: admin / admin123
3  普通用户: user / user123
```

Fence 6

测试数据

- 20只宠物 (狗、猫、其他)
- 15条公告
- 若干申请记录



常见问题

Q: 中文乱码？

A: 确保数据库字符集为 utf8mb4, JDBC URL 添加 ?

```
useUnicode=true&characterEncoding=utf8
```



更新日志

v1.0.0 (2026-07-02)

- ☒ 基础 CRUD 功能完成
- ☒ Swing 图形界面实现
- ☒ 管理员审核流程
- ☒ 级联删除支持
- ☒ 密码加密机制



贡献指南

欢迎提交 Issue 和 Pull Request!



许可证

MIT License



联系方式

项目维护者: 赵轩

邮箱: sympsel@outlook.com

感谢其他开发中提供帮助者:

- 袁一顺
- 周益同
- 王卓凡
- 李家祺

最后更新: 2026-07-02